

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-2 Créditos: 3	
---	---	---

Taller 2: Búsqueda local y juegos adversarios

Contexto general

La ciudad de Bogotá opera una red inteligente que distribuye energía y comunicaciones a hospitales, sistemas de movilidad, plantas de agua y centros de atención de emergencias. Después de una falla de gran escala, varios nodos críticos quedan funcionando con capacidad limitada y algunas secciones de la red presentan bloqueos, riesgo operativo y pérdida de cobertura.

Para iniciar la recuperación, el centro de operaciones dispone de un número limitado de módulos de estabilización. Estos módulos deben instalarse en sitios candidatos de manera que cubran los servicios prioritarios, no se concentren innecesariamente en la misma zona y eviten, cuando sea posible, los sectores de mayor exposición. En esta fase no interesa la secuencia de cambios usada para construir la solución, sino la calidad de la configuración final.

Mientras se restablece la red, un intruso intenta detener la operación. Un agente defensor debe desplazarse para activar terminales de protección antes de ser interceptado. Ambos agentes conocen el estado completo del entorno, actúan por turnos y persiguen objetivos opuestos: el defensor busca completar la activación y el intruso selecciona las respuestas que reducen sus posibilidades de éxito.

Objetivos de aprendizaje

Al completar este taller, el estudiante será capaz de:

1. Reconocer y analizar la formulación de un problema de optimización mediante configuraciones válidas, una función objetivo, una restricción y una relación de vecindad.
2. Implementar, comparar y evaluar ascenso de colina, recocido simulado y algoritmos genéticos, reconociendo máximos locales, mesetas y el efecto de la inicialización, los parámetros y la aleatoriedad sobre sus resultados.
3. Interpretar la formulación de un juego determinista de dos jugadores, suma cero, información perfecta y turnos alternados, e implementar Minimax con límite de profundidad.
4. Diseñar una función de evaluación para estados no terminales e implementar poda alfa-beta, verificando que conserva la decisión de Minimax y puede reducir la cantidad de nodos explorados.
5. Relacionar la topología de los mapas, la profundidad de búsqueda y las decisiones de los agentes, y analizar de manera argumentada las fortalezas y limitaciones de los algoritmos implementados.

Escenario del problema

El taller se divide en dos fases que comparten el contexto de la red, pero corresponden a dos problemas diferentes: optimización de configuraciones y búsqueda adversaria. Las fases son

computacionalmente independientes: la configuración producida en los puntos 1, 2 o 3 no constituye la entrada del juego de los puntos 4 y 5.

Fase 1: configuración de la red

Cada instancia define un conjunto de sitios candidatos N , nodos críticos c y un número fijo k de módulos disponibles. Una configuración se representa mediante una tupla binaria $(x_1, x_2, \dots, x_n)$, donde $x_i = 1$ indica que el sitio candidato i tiene instalado un módulo. Toda configuración válida debe contener exactamente k bits activos.

El puntaje combina tres componentes calculados por el proyecto:

$$\text{puntaje} = \text{cobertura} - \text{redundancia} - \text{exposición}$$

La cobertura recompensa la cercanía a nodos críticos y tiene en cuenta su prioridad. La redundancia penaliza módulos demasiado próximos que cubren prácticamente la misma región. La exposición penaliza el uso de sitios con riesgo operativo alto. El objetivo de los puntos 1, 2 y 3 es maximizar este puntaje usando la misma representación y las mismas instancias.

Una configuración vecina se obtiene retirando un módulo de un sitio seleccionado y ubicándolo en un sitio no seleccionado. Esta operación conserva exactamente k módulos. La generación de configuraciones válidas, vecinos, poblaciones iniciales, selección por torneo y reparación de cromosomas se entrega implementada.

Fase 2: defensa adversaria

El defensor y el intruso se desplazan sobre una cuadrícula. Las reglas generales son las siguientes:

- Ambos agentes pueden moverse arriba, abajo, izquierda, derecha o permanecer quietos.
- No pueden atravesar bloqueos ni salir de los límites del mapa.
- El defensor es el agente 0, corresponde a nodos MAX y mueve primero.
- El intruso es el agente 1, corresponde a nodos MIN y mueve después.
- Cuando el defensor entra a una terminal crítica pendiente, la activa.
- El defensor gana al activar todas las terminales y pierde si el intruso ocupa su misma posición.
- La victoria tiene utilidad $+1000$ y la derrota utilidad -1000 .
- Convención de profundidad: la profundidad se mide en *plies* o niveles del árbol. Cada acción de un agente consume un nivel. Por tanto, $depth = 1$ explora solamente una acción de MAX , mientras que $depth = 2$ incluye una acción de MAX y la respuesta de MIN . El estado raíz no consume profundidad.
- Durante las partidas, el intruso aplica realmente una política MIN : selecciona la acción de menor valor usando el mismo modelo, función de evaluación, profundidad y orden de acciones del experimento.

Los símbolos usados en los mapas son:

Símbolo	Leyenda	Fase	Elemento	Descripción
⊘		Ambas	Bloqueo	Celda no transitable que modifica las distancias y las rutas disponibles.
.		Ambas	Enlace normal	Celda transitable para el cálculo de distancias o el movimiento de los agentes.
N		Optimización	Sitio candidato	Posición donde puede instalarse un módulo de estabilización.
N		Optimización	Sitio candidato con exposición	Posición cuyo riesgo operativo aumenta la penalización de exposición cuando se instala un módulo.

Símbolo	Leyenda	Fase	Elemento	Descripción
C		Ambas	Nodo o terminal crítica	Servicio que debe recibir cobertura o terminal que debe activar el defensor.
D		Juego	Defensor	Posición inicial del agente MAX.
I		Juego	Intruso	Posición inicial del agente MIN.

Estructura del proyecto

El proyecto `RedCentinela` entrega una infraestructura funcional de carga de mapas, representación de configuraciones y estados, ejecución de algoritmos, simulación del intruso, métricas y visualización.

Archivos que debe modificar

Los siguientes archivos concentran la implementación solicitada en el taller. Mantenga los nombres de funciones, clases, parámetros y formatos de retorno definidos por el proyecto.

Archivo	Propósito	Funciones o métodos por completar	Puntos
<code>algorithms/optimization.py</code>	Contiene la función objetivo y los algoritmos de optimización.	<code>configuration_score</code> , <code>hill_climbing</code> , <code>cooling_schedule</code> , <code>simulated_annealing</code> , <code>one_point_crossover</code> , <code>swap_mutation</code> y <code>genetic_algorithm</code> .	1, 2 y 3
<code>algorithms/adversarial.py</code>	Define los agentes que toman decisiones sobre el árbol del juego.	<code>MinimaxAgent.get_action</code> y <code>AlphaBetaAgent.get_action</code> .	4 y 5
<code>algorithms/evaluation.py</code>	Contiene las funciones de evaluación utilizadas por Minimax y alfa-beta.	<code>evaluation_function</code> .	5

Otros módulos del proyecto

Los módulos siguientes hacen parte de la infraestructura. Puede consultarlos para comprender el funcionamiento del taller, pero no debe modificarlos para resolver los puntos solicitados.

Módulo o carpeta	Uso dentro del proyecto
<code>optimization/problem.py</code>	Carga las instancias, valida configuraciones y ofrece puntajes parciales, vecinos, poblaciones, selección y reparación.
<code>optimization/result.py</code>	Define el resultado común y las métricas retornadas por los algoritmos de optimización.
<code>optimization/runner.py</code>	Construye configuraciones iniciales y ejecuta el algoritmo seleccionado.
<code>world/layout.py</code>	Carga y valida los mapas adversarios y calcula distancias sobre la cuadrícula.
<code>world/game_state.py</code>	Representa el estado, las acciones legales, los sucesores, las terminales pendientes y las reglas de victoria o derrota.
<code>world/runner.py</code>	Ejecuta las partidas y aplica la política adversaria real del intruso como agente MIN.
<code>view/graphics_display.py</code>	Implementa la ventana gráfica emergente y la animación de resultados.
<code>view/text_display.py</code>	Implementa la visualización en texto.

Módulo o carpeta	Uso dentro del proyecto
view/display.py	Implementa el modo silencioso usado en experimentos.
main.py	Procesa los argumentos y ejecuta un experimento de optimización o una partida.
menu.py	Ofrece la interfaz de terminal para seleccionar punto, mapa, visualización y perfil. En optimización también permite definir repeticiones y semilla.
layouts/optimization	Contiene los mapas usados en los puntos 1, 2 y 3.
layouts/adversarial	Contiene los mapas usados en los puntos 4 y 5.

Ejecución

El proyecto requiere Python 3.11 o superior. Antes de ejecutar el taller, instale las dependencias desde la carpeta `RedCentinela`:

```
python -m pip install -r requirements.txt
```

La forma recomendada de ejecución es usar la interfaz de terminal:

```
python main.py
```

Desde esta interfaz puede seleccionar el punto del taller, el mapa, la visualización, la velocidad y el perfil de búsqueda. En los puntos 1, 2 y 3 también puede definir las repeticiones y la semilla.

La sintaxis general de ejecución por comandos es:

```
python main.py -m MODE -a ALGORITHM -l LAYOUT [OPCIONES]
```

Opciones de línea de comandos

Opción	Descripción	Valor inicial	Ejemplo
--help	Muestra la ayuda y termina el programa.	No aplica	python main.py --help
-m, --mode	Selecciona <code>optimization</code> o <code>adversarial</code> .	Obligatorio	-m optimization
-a, --algorithm	Selecciona el algoritmo o agente.	Obligatorio	-a simulatedAnnealing
-l, --layout	Selecciona el layout sin escribir la extensión.	Obligatorio	-l risk_corridor
--seed	Fija la semilla aleatoria en optimización.	7	--seed 12
-n	Define el número de ejecuciones en optimización.	1	-n 5
-t, -q, -x	-t activa el modo texto, -q el modo silencioso y -x define el tiempo entre cuadros de la animación gráfica.	Gráfica, 0.55	-t -x 0.1
--iterations	Limita las iteraciones de ascenso de colina o recocido.	500	--iterations 1000
--temperature, -cooling	Definen temperatura inicial y factor de enfriamiento.	20.0, 0.97	--temperature 30 -cooling 0.99
--population, --generations	Definen tamaño de población y generaciones.	40, 100	--population 60 --generations 150
--mutation, --elite	Definen probabilidad de mutación y cantidad de elites.	0.05, 2	--mutation 0.08 --elite 3
-d, --depth	Define la cantidad máxima de acciones o <i>plies</i> explorados desde la raíz.	2	-d 4
--max-rounds	Limita la duración de una partida.	60	--max-rounds 40

Ejemplos de comandos completos:

```
python main.py -m optimization -a hillClimbing -l local_peak -x 0.55
python main.py -m optimization -a simulatedAnnealing -l risk_corridor --seed 7
--temperature 20 --cooling 0.97 -t
python main.py -m optimization -a geneticAlgorithm -l balanced_city --
population 40 --generations 100 -q
python main.py -m adversarial -a MinimaxAgent -l tiny_defense -d 2 -x 0.55
python main.py -m adversarial -a AlphaBetaAgent -l medium_defense -d 4 --max-
rounds 40 -q
```

Ejercicios

Punto 1: Configuración inicial mediante ascenso de colina (15 %)

Después de la falla, el centro de operaciones necesita obtener rápidamente una configuración válida de módulos. Como primera estrategia utilizará ascenso de colina: desde una configuración inicial examinará todas las configuraciones vecinas y avanzará hacia la de mayor puntaje cuando represente una mejora estricta.

La función `problem.score_components(configuration)` retorna, en este orden, el beneficio de cobertura, la penalización por redundancia y la penalización por exposición. El algoritmo debe conservar el orden de los vecinos como regla de desempate, detenerse cuando no exista una mejora estricta o se alcance el límite de iteraciones y retornar un `OptimizationResult` con configuración, puntaje, evaluaciones, iteraciones e historial. No debe modificar la configuración inicial recibida.

- Presente una tabla breve que identifique la representación de una solución, la restricción de validez, la función objetivo y la relación de vecindad entregadas por el proyecto.
- Implemente `configuration_score` en `algorithms/optimization.py` usando cobertura - redundancia - exposición.
- Implemente `hill_climbing` en `algorithms/optimization.py` con la regla de mejora estricta indicada.
- Pruebe distintas configuraciones iniciales en `local_peak` y `plateau_grid`. Conserve evidencia de situaciones en las que el algoritmo alcance resultados distintos o se detenga sin encontrar el mejor valor observado.

Punto 2: Escape de óptimos locales mediante recocido simulado (15 %)

Los primeros experimentos muestran que ascenso de colina puede detenerse en una configuración mejor que todos sus vecinos inmediatos, pero inferior a otras regiones del espacio. Para aumentar la exploración se utilizará recocido simulado, que acepta siempre las mejoras y puede aceptar temporalmente movimientos que reducen el puntaje.

Para maximización, defina `delta = score(next) - score(current)`. Si `delta > 0`, acepte el movimiento. En caso contrario, acéptelo con probabilidad `exp(delta / temperature)`. La temperatura seguirá el programa geométrico $T(t) = T_0 * \alpha^t$.

- Implemente `cooling_schedule` en `algorithms/optimization.py`.
- Implemente `simulated_annealing` usando exclusivamente la fuente aleatoria recibida en `rng`. Debe elegir un vecino aleatorio, distinguir entre el estado actual y el mejor estado encontrado, registrar el historial y detenerse por temperatura mínima o límite de iteraciones.
- Selecione y compare al menos dos configuraciones representativas de temperatura inicial y factor de enfriamiento (`T0`, `alpha`) en uno o más layouts apropiados. Debido a la naturaleza

estocástica del algoritmo, realice varias ejecuciones cuando sea necesario para verificar que las diferencias observadas no correspondan únicamente a una corrida particular. Conserve resultados representativos para el punto 6.

Punto 3: Optimización poblacional mediante algoritmo genético (20 %)

Para explorar varias regiones del espacio al mismo tiempo, el centro de operaciones utilizará una población de configuraciones. Cada cromosoma conserva la representación binaria de los puntos anteriores y su aptitud corresponde a `configuration_score`.

El proyecto entrega la creación de poblaciones iniciales, la selección por torneo y la reparación de cromosomas. Su implementación debe mantener el tamaño de la población, preservar los mejores individuos indicados por `elite_size` y retornar el mejor individuo encontrado durante toda la ejecución, no únicamente el mejor de la última generación.

- a) Implemente `one_point_crossover` en `algorithms/optimization.py` y retorne dos descendientes.
- b) Implemente `swap_mutation` de la siguiente manera: con probabilidad `mutation_probability` por cromosoma, seleccione aleatoriamente un índice cuyo valor sea 1 y otro cuyo valor sea 0 e intercámbielos; en caso contrario, retorne el cromosoma sin modificación.
- c) Implemente `genetic_algorithm` integrando evaluación, selección por torneo, cruce, reparación, mutación, reemplazo generacional y elitismo. Todos sus operadores estocásticos deben usar exclusivamente la fuente `rng` recibida por el proyecto.
- d) Pruebe el algoritmo genético con al menos dos configuraciones representativas de sus parámetros en `risk_corridor`, `balanced_city` u otro layout apropiado. Seleccione configuraciones que permitan observar el efecto de al menos uno de sus parámetros principales. Analice brevemente cómo los parámetros modificados afectan el comportamiento del algoritmo. Conserve resultados representativos para el punto 6.

Punto 4: Defensa adversaria mediante Minimax (20 %)

El agente defensor debe activar las terminales mientras el intruso intenta interceptarlo. El defensor supone que el intruso siempre escogerá la respuesta de menor valor para MAX; por esta razón, una acción aparentemente favorable puede ser descartada cuando se anticipa la respuesta adversaria.

El método debe evaluar inmediatamente los estados terminales, usar `evaluation_function` cuando alcance el límite de profundidad, generar sucesores mediante `state.generate_successor` y respetar el orden retornado por `state.get_legal_actions`. La profundidad se mide en *plies*: cada acción usada para generar un sucesor reduce en uno la profundidad restante. Así, `depth = 1` permite anticipar una acción de MAX y `depth = 2` permite anticipar además la respuesta de MIN.

Para efectos de medición, `self.nodes_evaluated` debe reiniciarse al comienzo de cada llamada a `get_action` e incrementarse exactamente una vez por cada estado cuyo valor sea procesado por la búsqueda. El conteo incluye la raíz y los estados terminales o de corte por profundidad.

- a) Presente una tabla breve que identifique los seis componentes de la formulación entregada: estado inicial, jugador en turno, acciones, función de resultado o sucesor, prueba de estado terminal y utilidad.
- b) Implemente `MinimaxAgent.get_action` en `algorithms/adversarial.py` y retorne la acción seleccionada en la raíz.
- c) Modele al defensor, índice 0, como nodo MAX y al intruso, índice 1, como nodo MIN. En caso de empate conserve la primera acción según el orden entregado por el estado.
- d) Use `single_terminal` como control básico. Después compare profundidades de 2 y 4 plies en `tiny_defense`, `bottleneck` o `deceptive_distance`, registrando la acción inicial, el resultado y los nodos explorados.

Punto 5: Poda alfa-beta y función de evaluación (20 %)

Minimax explora ramas que no pueden modificar la decisión final. El centro de operaciones requiere un agente que conserve la misma acción y pueda reducir la cantidad de nodos examinados mediante poda. También necesita una función capaz de distinguir estados no terminales cuando el árbol se corta antes de llegar a una victoria o derrota.

- a) Implemente `AlphaBetaAgent.get_action` en `algorithms/adversarial.py`, manteniendo y actualizando correctamente `alpha` y `beta`. Use para `self.nodes_evaluated` la misma definición del punto 4.
- b) Complete `evaluation_function` en `algorithms/evaluation.py` sin modificar `base_evaluation_function`. Debe retornar +1000 para victoria y -1000 para derrota. Para todo estado no terminal debe retornar un valor estrictamente comprendido entre -1000 y +1000, de modo que ganar o perder siempre domine cualquier valoración heurística. Puede considerar el puntaje acumulado, terminales pendientes, distancia del defensor al objetivo, distancia del intruso al defensor, riesgo inmediato de captura y movilidad disponible.
- c) Para una misma entrada, profundidad, orden de acciones y evaluación, verifique que Minimax y alfa-beta seleccionen la misma acción. Compare la cantidad de nodos explorados por ambos algoritmos en `dual_terminal` y `medium_defense`.
- d) Justifique las características y los pesos de su función de evaluación y explique cómo estas permiten distinguir estados no terminales favorables y desfavorables para el defensor.

Punto 6: Reflexión sobre el trabajo realizado (10 %)

6a. Análisis de los algoritmos y resultados (6 %)

En el PDF de análisis, sintetice los resultados obtenidos durante los puntos 1 a 5. Utilice layouts y configuraciones representativas para sustentar sus conclusiones. Para los algoritmos estocásticos, realice varias ejecuciones cuando sea necesario para distinguir comportamientos consistentes de resultados particulares debidos a la aleatoriedad.

Para optimización, compare ascenso de colina, recocido simulado y algoritmo genético en términos de calidad de la solución, costo computacional y sensibilidad a la inicialización o a sus parámetros principales. Para búsqueda adversaria, compare Minimax y alfa-beta bajo condiciones equivalentes y considere la acción seleccionada, la profundidad y los nodos explorados.

Responda de manera argumentada:

- a) ¿Qué características de los layouts explican las diferencias observadas entre ascenso de colina, recocido simulado y algoritmo genético?
- b) ¿Qué efecto tuvieron la inicialización, la aleatoriedad y los parámetros principales sobre los resultados de los algoritmos de optimización?
- c) ¿Qué compromiso observa entre calidad de la solución y costo computacional en los algoritmos de optimización?
- d) Identifique un caso en el que cambiar la profundidad modifique la decisión o el resultado del defensor. Explique por qué ocurre y relacione su respuesta con la función de evaluación y el efecto horizonte.
- e) Explique por qué alfa-beta debe producir la misma decisión que Minimax bajo las mismas condiciones. Analice cómo la profundidad afecta el costo de la búsqueda y cómo el orden de las acciones afecta la eficacia de la poda alfa-beta.

6b. Reflexión sobre el uso de inteligencia artificial (4 %)

Escriba una reflexión concisa sobre el uso de IA durante el desarrollo de la solución final, de acuerdo con la política indicada más adelante. Explique qué aprendió al revisar sus sugerencias, si los cambios fueron correcciones fundamentales o mejoras secundarias, qué tan difícil fue formular prompts útiles y si podría explicar la solución final en una sustentación. Si no realizó ninguna iteración con IA, indique por qué no le pareció necesario ni útil. No hay respuestas correctas o incorrectas; este punto busca que reflexione sobre el uso de IA en tareas de programación.

Política de uso de inteligencia artificial

El uso de herramientas de inteligencia artificial deberá realizarse de forma responsable y documentada. Primero, los estudiantes deberán desarrollar una versión autónoma de la solución. Posteriormente, podrán utilizar herramientas de IA para apoyar la revisión, corrección, depuración, explicación o mejora del código.

Para cada función o fragmento trabajado con apoyo de IA, deberán incluir en los anexos del PDF la versión inicial correspondiente, los prompts utilizados, las respuestas o sugerencias relevantes, los cambios aceptados, rechazados o adaptados y una breve explicación de cómo verificaron su funcionamiento.

Los archivos del proyecto deberán contener únicamente la versión final activa del código. Los anexos deberán organizarse por punto, archivo o función, de manera que sea posible identificar con claridad el proceso seguido. Si no se utilizaron herramientas de IA, deberá indicarse explícitamente en el documento de reflexión.

Entrega del trabajo

- Guarde todo su trabajo, incluido el documento PDF de análisis, en una carpeta comprimida .zip.
- Suba el trabajo al espacio correspondiente en Bloque Neón a más tardar en la fecha y hora indicadas. Un solo integrante del grupo debe realizar la entrega.
- Incluya los nombres y códigos de todos los integrantes del grupo.
- No modifique las firmas de las funciones, los nombres de archivos ni los formatos de retorno definidos por el proyecto.

Nota: Al enviar su solución, usted declara que las versiones iniciales incluidas en los anexos son de su autoría, que las versiones desarrolladas con apoyo de IA corresponden a los prompts registrados, que comprende el funcionamiento del código final y que acepta que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.